

Executive summary

Current architecture

Login · Logout · Recovery

Trust boundaries

Findings by evidence class

Remediation backlog

Target architecture

Acceptance criteria

Open configuration questions

Appendix · Repository evidence

Crossangle Admin Auth Review

How the admin panel actually authenticates, where the real security boundary sits, and what must ship in the first security patch.

REPOSITORY	REVIEWED
E:\main · branch feature/dam-v3-milestone-planning	11 Sep 2026
SCOPE	
Login, logout, password recovery, session lifecycle, RBAC after auth	
METHOD	
Static trace of code, migrations, edge functions; no runtime probing of production	

1. Executive summary

The admin panel uses **Supabase Auth (GoTrue) behind a first-party cookie façade**. `POST /api/auth/login` exchanges email and password with GoTrue and sets two HTTP-only cookies. The browser's `supabase-js` client never holds a session: every data request travels through `/api/supabase/*`, where Vercel Edge middleware (production) or the Vite proxy (development) rewrites the `access_token` cookie into an `Authorization` header. Roles live in `public.user_roles`. Server-side authorization is Postgres RLS via `has_role`, `is_admin_or_editor` and `is_platform_admin`, plus per-function checks inside Deno edge functions.

The design intent is sound and is guarded by a CI invariant. The implementation, however, shipped in a partially-finished state. Three defects are confirmed directly from repository code:

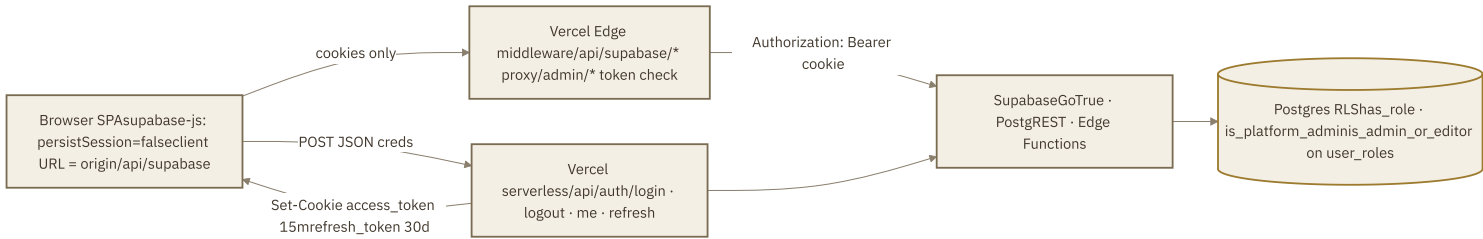
- **The client role is hardcoded to `super_admin`** on every page load; the real lookup is commented out (committed 5 Sep 2026). Every authenticated account sees the full admin UI.
- **The login response body returns the access and refresh tokens as JSON**, which defeats the purpose of the HTTP-only cookies.
- **Password recovery cannot complete**. Nothing consumes the recovery tokens GoTrue places in the redirect URL, so the password update runs without a session.

Conclusion. P0 remediation begins with the hardcoded role, but the token exposure and the recovery defect must be fixed *in the same security patch*. Fixing only `AuthProvider.tsx:157` leaves two serious authentication vulnerabilities intact.

Do not rely on UI authorization. `AuthGuard` and `RoleGuard` shape navigation; they are not a security control and never were. The security boundary is Postgres RLS on `user_roles`-backed policies and the `getUser` + role check inside each edge function. Restoring `RoleGuard` fixes the user experience and closes an information-exposure gap in the UI; it does not, by itself, change what any account can read or write.

2. Current architecture

RUNTIME COMPONENTS AND THE SINGLE REAL AUTHORIZATION BOUNDARY



Two things worth holding onto: the middleware's `/admin/*` check verifies that a token is *valid*, never that it belongs to an admin; and because the SPA client has no in-memory session, every GoTrue-side operation that depends on one (recovery, revocation) is silently a no-op unless a cookie is present.

3. Login, logout and recovery sequences

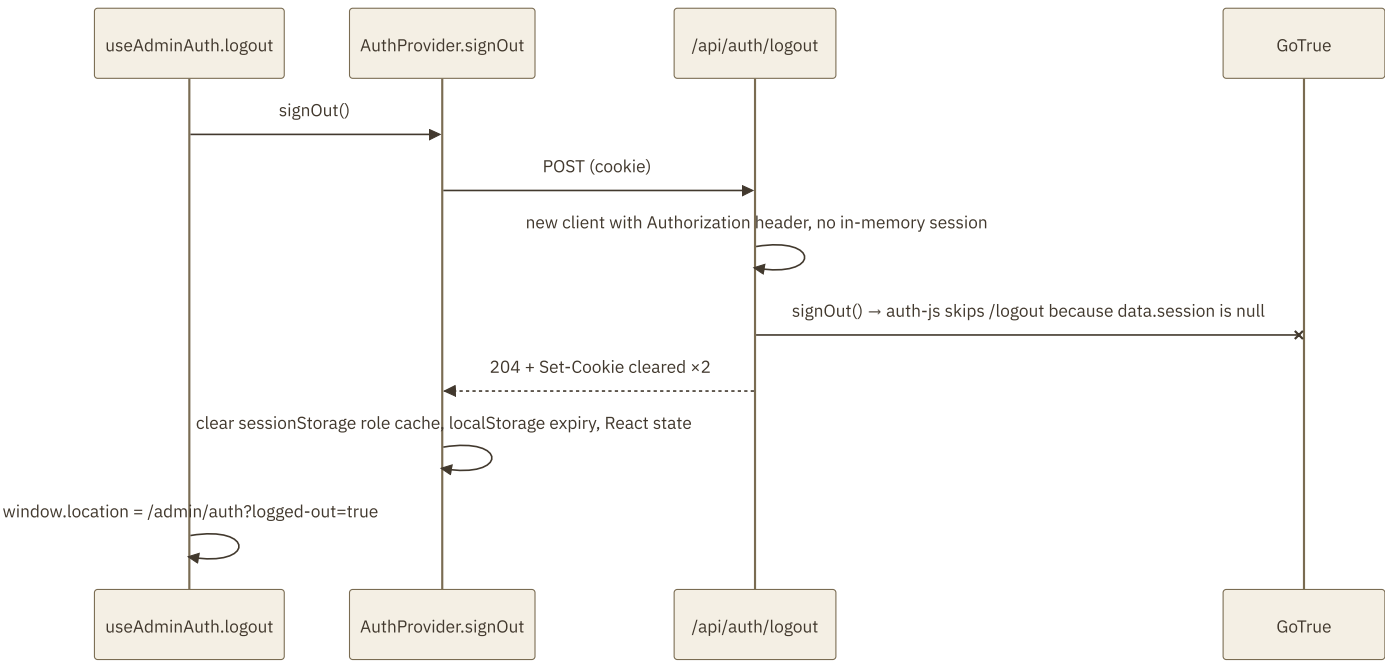
Login

AS IMPLEMENTED

```
sequenceDiagram
    participant A as Admin
    participant UI as AdminAuth.tsx
    participant API as /api/auth/login
    participant G as GoTrue
    participant DB as user_roles (service key)
    participant P as AuthProvider
    A->>UI: email + password (HTML required / type=email only)
    UI->>API: POST JSON
    API->>G: signInWithPassword
    G-->>API: access JWT + refresh token
    API->>DB: select role where user_id
    API-->>UI: Set-Cookie x2 + JSON {user, role, session:{access_token, refresh_token}}
    UI->>UI: window.location = ROLE_DEFAULT_ROUTE[role]
    Note over UI,G: Edge middleware calls GoTrue /user for /admin/* documents
    P->>API: GET /api/auth/me
    API->>G: getUser(cookie token)
    P->>P: role = "super_admin" (hardcoded; fetchUserRole commented out)
```

Logout

AS IMPLEMENTED — NOTE THE MISSING REVOCATION



Password recovery

AS IMPLEMENTED — THE TOKENS IN THE FRAGMENT ARE NEVER READ

```
sequenceDiagram
    participant A as User
```

```
participant UI as AdminAuth.tsx
participant G as GoTrue (via /api/supabase proxy)
A->>UI: email
UI-)G: POST /auth/v1/recover (fire-and-forget)
UI-->>A: "Check your email" after 600 ms, always
G-->>A: email link → /admin/auth#type=recovery&access_token=...
A->>UI: opens link
Note over UI: detectSessionInUrl=false; no setSession / exchangeCodeForSession
A->>UI: new password (client policy ≥8, upper, digit, symbol)
UI->>G: PUT /auth/v1/user
alt no cookie
    G-->>UI: 401 → UI shows "link expired"
else admin already logged in
    Note over G: middleware injects cookie token → updates the logged-in account, not t
end
```

4. Trust boundaries

BOUNDARY	TRUSTED	UNTRUSTED	VALIDATED BY	AUTHORIZATION ENFORCED BY	CLIENT-BYPASSABLE
Browser → /api/auth/login	Serverless handler, env keys	Request body	Presence check only; GoTrue verifies the password	None (authentication only)	n/a
Browser → /api/supabase/*	Edge middleware, cookie	Path, body, all other headers	GoTrue verifies the JWT per request	Postgres RLS — the real boundary	No (server-side)
Edge middleware → /admin/* documents	GoTrue /user response	Cookie	Token validity	None — no role check	Irrelevant: SPA shell is public anyway
AuthGuard / RoleGuard	—	—	React state	UI only	Yes, by definition
Edge functions (manage-user, sync-user-role, assign-first-admin, ...)	Service-role client	Bearer token, body	auth.getUser(token) then user_roles lookup	Function code	No
login.ts role lookup	Service-role key (bypasses RLS)	—	—	Result is informational: used only for the redirect target	n/a

5. Findings by evidence class

Every finding carries an evidence label. **Verified** means read directly from repository code or config. **Inferred** means the code makes it near-certain but runtime behaviour was not executed against production. **Unknown** means the answer lives in Supabase dashboard configuration that the repository does not contain (supabase/config.toml has no [auth] section).

VERIFIED from code / config / tests **INFERRED** strongly implied, not executed

UNKNOWN not establishable from the repo

Client role is hardcoded to super_admin	
EVIDENCE	apps/web/src/components/auth/AuthProvider.tsx:157 — const userRole = "super_admin" as AppRole; // await fetchUserRole(serverUser.id); .Present since commit ea1e1b35 (2026-09-05).
IMPACT	Editors, viewers and accounts with <i>no</i> role row render every admin module, including user and role management screens. Data access is still bounded by RLS, so this is UI exposure and a functional defect, not a data breach on its own.
ATTACK	Any account that can pass /api/auth/me (any GoTrue user — see F-09) navigates the full admin surface and can attempt every mutation, relying on RLS alone to refuse.

F-02	HIGH	VERIFIED
Login response returns access and refresh tokens in JSON		
EVIDENCE	apps/web/api/auth/login.ts:79-87 returns session: { access_token, refresh_token } . The client reads only data.role (AdminAuth.tsx:158).	
IMPACT	A 30-day refresh token is exposed to page JavaScript, browser extensions, and any response logging. This nullifies the HTTP-only cookie model the CI invariant exists to protect.	
ATTACK	Stored or reflected XSS anywhere on the origin hooks fetch and exfiltrates the refresh token at login; the attacker mints access tokens for 30 days regardless of logout (see F-06).	

F-03	HIGH	VERIFIED	RUNTIME INFERRED
Password recovery tokens are never consumed			
EVIDENCE	client.ts:19-21 sets detectSessionInUrl: false ; a repository-wide search finds no setSession , exchangeCodeForSession or verify0tp that reads the recovery URL. AdminAuth.tsx only inspects #type=recovery to switch views.		
VERIFIED	No code path can establish a recovery session.		
INFERRED	Logged-out users receive 401 from PUT /auth/v1/user and see "link expired". If an admin is already signed in, the middleware injects <i>their</i> cookie token, so the password of the logged-in account is changed instead of the recovery target's.		
IMPACT	Recovery is non-functional; the logged-in edge case can change the wrong account's password. Accounts that lose their password are locked out until a super admin intervenes.		

F-04	MEDIUM	INFERRED
Possible self-escalation via profiles.role → sync-user-role		
EVIDENCE	Migration 20260702000000_rbac_v2_editor_role.sql §9: admin_update_profiles allows auth.uid() = id with no column restriction. supabase/functions/sync-user-role/index.ts:125-140 reads profiles.role with the service role and writes it into user_roles when no row exists. A profiles_role_check constraint exists (20260317000003) but its permitted values were not confirmed.	
WHY INFERRED	Migrations are a partial changelog (ADR 0003); the live policy set may differ. The check constraint may already exclude admin values.	
ATTACK	An authenticated account with no user_roles row updates its own profiles.role to admin , then invokes sync-user-role . If the constraint permits the value, it becomes a platform admin at the RLS layer — a genuine privilege escalation, not a UI one.	

F-05	MEDIUM	VERIFIED
No caller for /api/auth/refresh ; access cookie expires at 15 minutes		
EVIDENCE	login.ts:62 sets maxAge: 15 * 60 ; refresh.ts exists but zero references in src/ . autoRefreshToken: false in client.ts .	
IMPACT	After 15 minutes the proxy forwards requests with no Authorization ; PostgREST treats them as anon . The UI shows empty tables and failed saves with no redirect until reload. The 30-day refresh cookie is dead weight. Reliability rather than confidentiality, but it drives users to unsafe workarounds.	

F-06	MEDIUM	VERIFIED
Logout clears cookies but revokes nothing server-side		
EVIDENCE	logout.ts builds a client with a global Authorization header and calls signOut() .	

	@supabase/auth-js 2.X _signOut (GoTrueClient.js:3418-3420) only calls /logout when an <i>in-memory</i> session exists; this client has none.
IMPACT	The refresh token remains valid for its lifetime and the access JWT until exp . Combined with F-02, a captured token survives logout. Other tabs keep stale React state until their next request fails.

F-07	MEDIUM	VERIFIED	LIMITS UNKNOWN
No admin-side logout or rate limiting on login			
EVIDENCE	login.ts has no throttling. supabase/functions/admin-login contains KV-backed logout logic but is referenced nowhere in src/ — dead code.		
UNKNOWN	GoTrue's per-IP sign-in limits are dashboard-configured and not in the repo.		
IMPACT	Credential stuffing against admin emails is bounded only by whatever GoTrue defaults are in force.		

F-08	LOW	VERIFIED	
No CSRF / Origin check on proxied mutations			
EVIDENCE	middleware.ts:333-373 forwards any method to Supabase with the cookie token injected; no Origin or Sec-Fetch-Site check. Cookies are SameSite=Lax .		
IMPACT	Lax blocks cross-site POST in modern browsers. Residual exposure is GET-invocable RPCs reached via top-level navigation, and any future relaxation of SameSite.		

F-09	LOW	VERIFIED	SIGNUP STATE UNKNOWN
Authentication endpoints never consult user_roles			
EVIDENCE	me.ts returns 200 for any valid GoTrue user; middleware verifyAuth checks token validity only.		
UNKNOWN	Whether public signup is enabled on the project. If it is, F-01 turns any self-registered account into a full-UI admin (still RLS-bounded).		

F-10	LOW	VERIFIED	
Hygiene: verbatim GoTrue errors, loose cookie regex, no auth audit trail			
EVIDENCE	login.ts:31 returns error.message unchanged ("Email not confirmed" confirms an account exists; "Invalid login credentials" is uniform). All handlers parse cookies with /access_token= ([^;]+) / . No login/logout/reset events are written to system_logs or any audit table.		
IMPACT	Minor enumeration; brute-force and takeover are undetectable after the fact.		

What is already right

VERIFIED STRENGTHS - No client-side token storage; persistSession , autoRefreshToken , detectSessionInUrl all false , enforced by scripts/checks/supabase-auth-invariant.js on pre-commit and CI. - Cookies are HTTP-only, Secure in production, SameSite=Lax ; refresh cookie is path-scoped to /api/auth/refresh . - Role lookup at login uses the service key, so RLS on user_roles cannot hide the row. - RLS helpers are SECURITY DEFINER with pinned search_path ; user_roles writes are super-admin only. - Forgot-password view is enumeration-safe: fire-and-forget request, constant 600 ms response, 60 s resend cooldown. - Edge functions verify the bearer token and role in code rather than trusting the gateway.	NOT A FINDING, BUT NOTE "Remember me" is a 24 h localStorage timer read by AuthProvider ; it is a UX convenience, not a session policy. - AuthProvider calls supabase.auth.setSession with an empty refresh token; auth-js rejects it (AuthSessionMissingError), so the call is a harmless no-op consistent with the cookie model. - Client password policy (≥8, uppercase, digit, symbol) exceeds NIST 800-63B's recommendation against composition rules; length plus breach-list checks would serve better.
--	---

6. Remediation backlog – ranked by exploitability

Order reflects how easily an attacker or an ordinary user reaches the defect today, not raw CVSS-style severity. Items 1–3 are one patch.

-
- 1

Restore the real role fetch and fail closed

Replace the literal at `AuthProvider.tsx:157` with `await fetchUserRole(serverUser.id) ; treat null as denied and route to /admin/auth?error=role_unavailable` , which the login page already handles. Add a fitness test forbidding `as AppRole` literal assignments under `components/auth` .

SECURITY PATCH 1 · CLOSES F-01
-
- 2

Strip tokens from the login response

Return `{ user, role }` only. The client never used `session` . Extend `supabase-auth-invariant.js` to assert no handler under `api/auth` serialises `refresh_token` into a body.

SECURITY PATCH 1 · CLOSES F-02
-
- 3

Make recovery work through the server

Add `POST /api/auth/recover` : receive the recovery `code` (switch `resetPasswordForEmail` to PKCE) or the fragment tokens, exchange them with GoTrue server-side, call `updateUser` with that session, then global sign-out and clear cookies. Never let the middleware's cookie injection touch this path.

SECURITY PATCH 1 · CLOSES F-03
-
- 4

Close the `profiles.role` escalation path

Confirm live `profiles_role_check` values; add a trigger or column privilege so authenticated users cannot change their own `profiles.role` ; make `sync-user-role` only ever write `viewer` from a profile, never `admin` or above.

PATCH 2 · CLOSES F-04
-
- 5

Refresh the session instead of letting it decay

Have the `/api/supabase` Edge proxy (or a thin fetch wrapper) retry once through `/api/auth/refresh` on 401, and align the access cookie `maxAge` with the JWT `exp` . Surface "session expired" in the UI when refresh fails.

PATCH 2 · CLOSES F-05
-
- 6

Revoke on logout

In `logout.ts` , call `POST {SUPABASE_URL}/auth/v1/logout?scope=global` directly with the cookie token before clearing cookies. Broadcast logout across tabs with `BroadcastChannel` .

PATCH 2 · CLOSES F-06
-
- 7

Rate-limit and log authentication

Per-IP and per-email throttling on `/api/auth/login` (Vercel KV / Upstash), generic error messages, and success/failure/reset/logout events into `system_logs` . Delete or wire the dead `admin-login` function.

PATCH 3 · CLOSES F-07, F-10
-
- 8

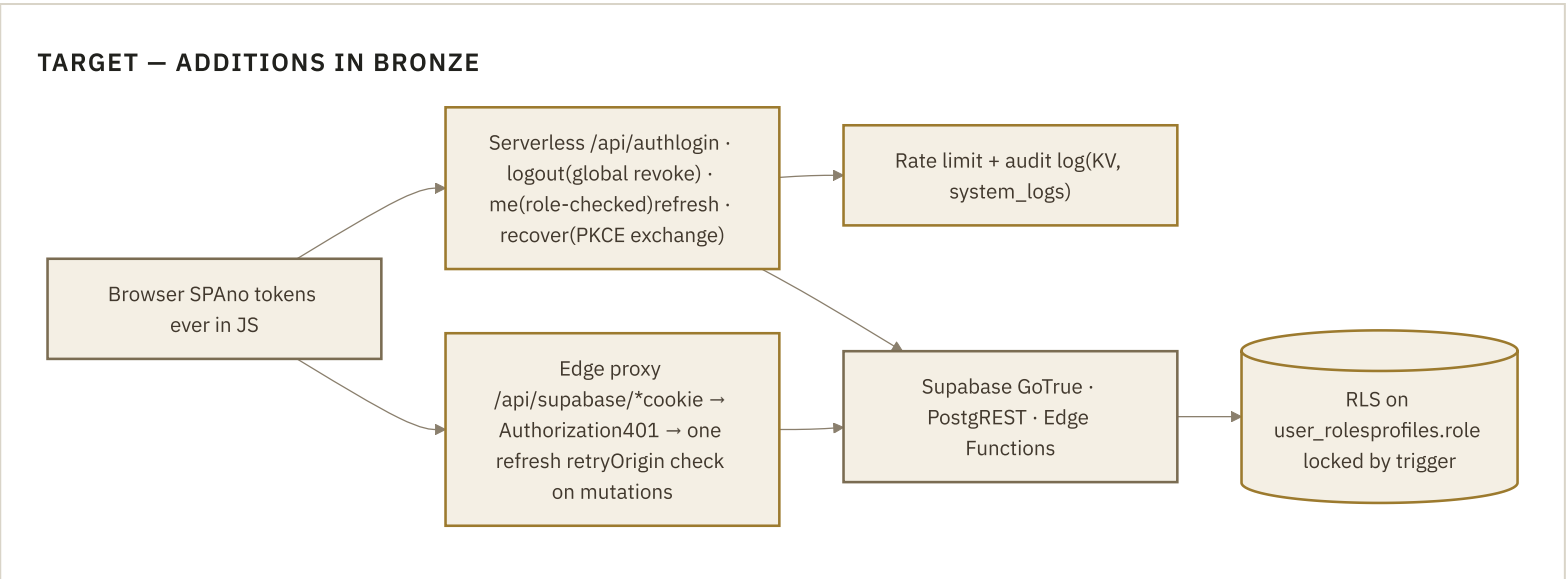
Deny non-staff at the authentication layer

`/api/auth/me` (or the middleware) returns 403 when the user has no `user_roles` row; add an `Origin / Sec-Fetch-Site` check on non-GET proxy requests.

PATCH 3 · CLOSES F-08, F-09
-

7. Proposed target architecture

The target keeps the cookie façade and RLS boundary — they are the right shape — and completes the half-built lifecycle. Nothing here introduces a new provider or framework.



ESTABLISHED PRACTICE APPLIED • Tokens confined to HTTP-only cookies; nothing token-shaped in response bodies (OWASP Session Management). • Server-side revocation on logout and after password change (ASVS V3.3). • Recovery via single-use, server-exchanged code; generic responses (OWASP Forgot Password). • Throttling on the credential endpoint (ASVS V2.2).	OPTIONAL, LATER • MFA for <code>super_admin</code> (GoTrue TOTP is available; UI work required). • Move cookie issuance into the Edge runtime to consolidate on one runtime. • Replace composition rules with length + breached-password check.
---	--

8. Test and acceptance criteria

Existing coverage: `e2e/admin-full-flow.spec.ts` (login, logout redirect, unauthenticated redirect) and `scripts/checks/supabase-auth-invariant.js` (client configuration). The following are the acceptance gates for the backlog above; a fix is not done until its row passes.

SCENARIO	TEST	PASSES WHEN	CLOSES
Role resolution	Vitest on <code>AuthProvider</code> with mocked <code>/api/auth/me</code> and <code>user_roles</code>	<code>role</code> equals the DB row; null row renders no admin route	F-01
Role guard regression	<code>architecture.test.ts</code> contract	No <code>as AppRole</code> literal assignment under <code>components/auth</code>	F-01
Unauthorized admin access	Playwright as <code>editor</code> and <code>viewer</code>	<code>/admin/system</code> , <code>/admin/user-access</code> redirect to the role's default route	F-01
Direct API access	Fetch <code>/api/supabase/rest/v1/user_roles</code> as <code>editor</code>	RLS returns only own row; insert/update refused	Boundary
Token exposure	API test on <code>/api/auth/login</code>	Body contains no <code>access_token</code> / <code>refresh_token</code> keys	F-02
Recovery happy path	Playwright against a staging project mailbox	Link → new password → login succeeds; old password fails	F-03
Recovery token reuse / expiry	Replay the same code; use an expired code	Both return 4xx and the "expired" view	F-03
Recovery while logged in as another admin	Playwright with an admin cookie present	The recovery target's password changes, the admin's does not	F-03

Privilege escalation	As no-role user: PATCH profiles role=admin, invoke sync-user-role	Update refused or role stays ≤ viewer	F-04
Expired session	Clock-skip 16 min, then a data request	Request succeeds after transparent refresh; failed refresh shows "session expired"	F-05
Logout invalidation	Capture cookie, logout, replay cookie against PostgREST	401	F-06
Concurrent tabs	Logout in tab A	Tab B redirects within one tick	F-06
Brute force	N failed logins from one IP	429 before N reaches the threshold; audit rows written	F-07, F-10
Account enumeration	Unknown email vs wrong password	Identical status, body, and timing envelope	F-10
Session fixation	Pre-set an access_token cookie, then log in	Cookie is replaced by a freshly issued token	Boundary

9. Open configuration questions

These cannot be answered from the repository and change the severity of several findings. Confirm them in the Supabase dashboard before finalising the patch plan.

Is public signup enabled on the project?					UNKNOWN	–
If yes, F-01 and F-09 compound: any self-registered account reaches the full admin UI. If no, F-01 is confined to existing staff accounts.						
JWT expiry and refresh-token rotation settings					UNKNOWN	+
Recovery OTP / magic-link expiry and single-use enforcement					UNKNOWN	+
GoTrue sign-in rate limits					UNKNOWN	+
Live	profiles_role_check	constraint values and live	profiles	policies	UNKNOWN	+
Password policy configured in GoTrue					UNKNOWN	+

10. Appendix – repository evidence

PATH	RESPONSIBILITY	KEY REFERENCES
apps/web/api/auth/login.ts	Password sign-in, service-key role lookup, cookie issuance	:31 verbatim error · :37-55 role · :58-75 cookies · :79-87 token leak
apps/web/api/auth/logout.ts	Cookie clearing; intended revocation	signOut without session (no-op)
apps/web/api/auth/me.ts	Cookie → getUser ; no role check	returns refresh_token: ""
apps/web/api/auth/refresh.ts	Refresh-token	zero callers in src/

	rotation	
apps/web/middleware.ts	Edge proxy, admin token check, mobile block, OG injection	:175-193 verifyAuth · :333-373 proxy · :375-383 admin gate
apps/web/vite.config.ts	Dev emulation of proxy; local execution of api/auth/*	:39 vercelApiPlugin · :189-203 proxy
apps/web/src/integrations/supabase/client.ts	Browser client against /api/supabase	:19-21 auth options
apps/web/src/components/auth/AuthProvider.tsx	Client auth state, role, signOut	:157 hardcoded role · :107 setSession no-op
apps/web/src/components/auth/AuthGuard.tsx , components/admin/RoleGuard.tsx	UI route gating	—
apps/web/src/components/auth/_internals/useRoleFetcher.ts , role-cache.ts	Role lookup with retry and 5-min sessionStorage cache	currently bypassed on initial load
apps/web/src/pages/admin/AdminAuth.tsx	Login / forgot / reset / expired / logged-out views	:127 login · :186-225 forgot · :228-260 reset
apps/web/src/hooks/useAdminAuth.ts	Logout trigger and hard navigation	—
apps/web/src/lib/auth/rbac.ts , routes/adminRoutes.tsx	Role model; per-route allow-lists	adminRoutes :52-139
supabase/migrations/20260103223827_*.sql , 20260326000000_admin_user_roles_v1.sql , 20260702000000_rbac_v2_editor_role.sql	user_roles , RLS helpers, policies	rbac_v2 §5 is_platform_admin · §9 profiles policies
supabase/functions/sync-user-role , manage-user , assign-first-admin	Server-side role operations with in-code token verification	sync-user-role :125-140
supabase/functions/admin-login	KV-backed lockout login — unreferenced	dead code
apps/web/scripts/checks/supabase-auth-invariant.js	CI gate for the cookie model	pre-commit + CI
e2e/admin-full-flow.spec.ts , e2e/setup/admin-auth.ts	Only existing auth tests	login, logout redirect, unauthenticated redirect
node_modules/@supabase/auth-js/dist/main/GoTrueClient.js	Library behaviour cited for F-03 and F-06	:3019 setSession guard · :3418 signOut guard

partial migration history); low for GoTrue configuration (not in the repository).